SIMATS ENGINEERING
ASSESSMENT TOOL 1-APPLICATION- BASED QUESTIONS

COURSE NAME: Cryptography and Network Security
COURSE CODE: CSA5103

COURSE OUTCOME COVERED

CO1: Apply classical encryption techniques using symmetric and asymmetric ciphers to encrypt and decrypt data for the ensuring data security. (BL3, BL4)

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Total Marks:100**

Annexure A

APPLICATION- BASED QUESTIONS

Code of Conduct:

*I **RAKSHITH R.R (192324156)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

APPLICATION- BASED QUESTIONS

Scenario: Event Communication Security A student organization is preparing to share confidential event details with its core members using a simple classical encryption method. During testing, a message "MEET AT NOON" is transformed into an encoded version using a fixed shift pattern.
Reconstruct how the encrypted message might appear if a shift of 4 positions is applied.
Explain how an authorized member, aware of the encoding rule, would retrieve the original message.

Scenario: Intercepted Suspicious Message A cybersecurity intern intercepts the coded message: "GSRH RH Z HVXIVG" during network monitoring. The structure of the text suggests a simple substitution mechanism rather than complex encryption.
Deduce the most likely original message.
Justify your answer by analyzing repeating patterns, letter structures, or known cipher characteristics.

Scenario: Secure Messaging System A company deploys a keyword-based encryption technique for internal communication. An employee sends the word "HELLO" using the keyword "KEY" to ensure confidentiality.
Demonstrate how the encryption process transforms the message step-by-step.
Explain how the repeating keyword influences each character of the ciphertext.

Scenario: Suspicious Digital Asset Investigation During a digital forensic audit, investigators suspect that a company logo file may contain hidden confidential information embedded within it. The file appears normal upon casual inspection.
Analyze how investigators could detect the presence of hidden data within the image.
Recommend two techniques that could strengthen the security of such hidden communications against detection or extraction.

Scenario: Legacy Encryption in Corporate Communication A legacy system in a company still uses a transposition-based encryption method with three levels of arrangement. A received message appears as: "WECRLTEERDSOEEFEAOCAIVDEN".

a. Determine the most probable original message by reversing the encryption process.
b. Critically evaluate why such a method is vulnerable in modern communication systems.
c. Propose one practical enhancement to improve the confidentiality of messages in this scenario.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding g; limited or partial integration	Poor understanding g; unable to integrate concepts
Experiment al Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation on with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

SIMATS ENGINEERING

ASSESSMENT TOOL 1 – APPLICATION-BASED QUESTIONS

Solution Report

1. Overview and Tools

Both questions deal with classical symmetric ciphers. Question 1 uses a substitution cipher, where every letter is replaced by another letter, and Question 5 uses a transposition cipher, where letters keep their identity but change position. Each answer follows the same structure: concept, experimental design (method), tools and implementation, and results with analysis.

Tool / Technique	Purpose
Manual calculation	Step-by-step modular arithmetic and rail-grid construction, shown in tables.
Python 3	Implementation of Caesar and Rail Fence encrypt / decrypt functions; brute-force testing of every possible key.
Python cryptography library	AES-256-GCM demonstration for the proposed enhancement in Question 5(c).

2. Question 1 – Event Communication Security

Problem: Encrypt the message “MEET AT NOON” with a fixed shift of 4 positions, and explain how an authorized member retrieves the original message.

2.1 Concept Understanding

A shift (Caesar) cipher is a mono-alphabetic substitution cipher. The 26 letters are numbered A = 0 to Z = 25. Encryption adds the key k to each letter number and wraps around the alphabet using arithmetic modulo 26. Decryption subtracts k, again modulo 26. It is a symmetric cipher because the sender and the receivers share the same secret key (k = 4).

Encryption: $C = (P + k) \bmod 26$ Decryption: $P = (C - k) \bmod 26$

2.2 Experimental Design

Step 1 – build the shifted alphabet for k = 4 (Figure 1). Step 2 – convert each letter to its number, add 4 and reduce modulo 26. Step 3 – regroup the letters in the original word pattern. Step 4 – verify by decrypting the result and by running the same logic in Python.

	A	**B**	**C**	**D**	**E**	**F**	**G**	**H**	**I**	**J**	**K**	**L**	**M**
Plain	A	B	C	D	E	F	G	H	I	J	K	L	M
Cipher	E	F	G	H	I	J	K	L	M	N	O	P	Q

	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Plain	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Cipher	R	S	T	U	V	W	X	Y	Z	A	B	C	D

Figure 1: Plain alphabet and its cipher alphabet for a shift of 4 (A→E, B→F, … W→A, X→B, Y→C, Z→D)

2.3 Encryption (Sender Side)

Word	Plain letter	P (A=0)	(P + 4) mod 26	Cipher letter
MEET	M	12	12 + 4 = 16	**Q**
MEET	E	4	4 + 4 = 8	**I**
MEET	E	4	4 + 4 = 8	**I**
MEET	T	19	19 + 4 = 23	**X**
AT	A	0	0 + 4 = 4	**E**
AT	T	19	19 + 4 = 23	**X**
NOON	N	13	13 + 4 = 17	**R**
NOON	O	14	14 + 4 = 18	**S**
NOON	O	14	14 + 4 = 18	**S**
NOON	N	13	13 + 4 = 17	**R**

Table 1: Letter-by-letter encryption of "MEET AT NOON"

Encrypted message: QIIX EX RSSR (MEET → QIIX, AT → EX, NOON → RSSR)

2.4 Decryption (Authorized Member)

An authorized member knows the rule "shift of 4". He or she simply reverses it: subtract 4 from every cipher letter number, adding 26 whenever the result is negative (wrap-around). Spaces are kept as word separators.

Word	Cipher letter	C (A=0)	(C − 4) mod 26	Plain letter
QIIX	Q	16	16 − 4 = 12	**M**
QIIX	I	8	8 − 4 = 4	**E**
QIIX	I	8	8 − 4 = 4	**E**
QIIX	X	23	23 − 4 = 19	**T**
EX	E	4	4 − 4 = 0	**A**
EX	X	23	23 − 4 = 19	**T**
RSSR	R	17	17 − 4 = 13	**N**
RSSR	S	18	18 − 4 = 14	**O**
RSSR	S	18	18 − 4 = 14	**O**
RSSR	R	17	17 − 4 = 13	**N**

Table 2: Letter-by-letter decryption of "QIIX EX RSSR"

Recovered message: MEET AT NOON

Wrap-around example: if the cipher letter were C (2), then 2 − 4 = −2, and −2 + 26 = 24, which is Y. In the message above no wrap-around occurs during decryption because every cipher letter is at least 4 positions

from A.

2.5 Results and Analysis

A person who does not know the key can try all 25 possible shifts (brute force). Only one gives readable English, so the cipher is broken in seconds:

Shift tried	Decrypted text	Meaningful?
1	PHHW DW QRRQ	No
2	OGGV CV PQQP	No
3	NFFU BU OPPO	No
4	MEET AT NOON	**YES – correct key**
5	LDDS ZS MNNM	No
6	KCCR YR LMML	No
7	JBBQ XQ KLLK	No
8	IAAP WP JKKJ	No

Table 3: Brute-force attack on the cipher text (shifts 1–8 shown)

Observations: (i) the key space is only 25, so the cipher offers no real protection against a computer; (ii) the word structure 4-2-4 is preserved, which reveals word lengths and helps guessing; (iii) each plain letter always maps to the same cipher letter, so frequency analysis also works (the repeated pairs II and SS reveal the double letters EE and OO). The method is, however, acceptable as a classroom demonstration of symmetric key encryption, and the output was cross-verified: decryption of QIIX EX RSSR with k = 4 returns MEET AT NOON in both the manual and the Python runs.

3. Question 5 – Legacy Encryption in Corporate Communication

Problem: A transposition cipher with three levels of arrangement produced the cipher text WECRLTEERDSOEEFEAOCAIVDEN. (a) Recover the original message, (b) evaluate why the method is vulnerable today, (c) propose one practical enhancement.

3.1 Concept Understanding

A transposition cipher rearranges the plaintext letters without changing them. “Three levels of arrangement” points to the Rail Fence (zig-zag) cipher with 3 rails: the plaintext is written diagonally down and up across three rows, and the cipher text is obtained by reading the rows one after another (row 1, then row 2, then row 3). The key is the number of rails, here 3.

3.2 (a) Reversing the Encryption – Experimental Design

Step 1 – count the cipher text: 25 letters. Step 2 – lay out the zig-zag pattern for 25 positions (rows 1-2-3-2-1-2-3-2-1 …); this tells how many letters belong to each rail: rail 1 has 7, rail 2 has 12 and rail 3 has 6 (7 + 12 + 6 = 25). Step 3 – cut the cipher text into those three segments and place each segment back on its rail. Step 4 – read the grid in zig-zag order from position 1 to 25.

Rail	Positions occupied (zig-zag 1-2-3-2-1…)	Letters	Cipher segment
1	1, 5, 9, 13, 17, 21, 25	7	WECRLTE
2	2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24	12	ERDSOEEFEAOC
3	3, 7, 11, 15, 19, 23	6	AIVDEN
Total		**25**	WECRLTE|ERDSOEEFEAOC|AIVDEN

Table 4: Splitting the cipher text into rails

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
Rail 1	**W**				**E**				**C**				**R**				**L**				**T**				**E**
Rail 2		**E**		**R**		**D**		**S**		**O**		**E**		**E**		**F**		**E**		**A**		**O**		**C**	
Rail 3			**A**				**I**				**V**				**D**				**E**				**N**		

Figure 2: Reconstructed 3-rail zig-zag grid (reading the letters from left to right gives the plaintext)

Recovered plaintext: WEAREDISCOVEREDFLEEATONCE

Most probable original message: "WE ARE DISCOVERED. FLEE AT ONCE."

Validation

Re-encrypting WEAREDISCOVEREDFLEEATONCE with 3 rails in Python gives exactly WECRLTEERDSOEEFEAOCAIVDEN, which proves the result. Trying other rail counts gives meaningless text, so 3 rails is the only valid key:

No. of rails tried	Result of reversing	Meaningful?
2	WEEFCERALOTCEAEIRVDDSEONE	No
3	**WEAREDISCOVEREDFLEEATONCE**	**YES**
4	WTEVFEEEEDARCDOECSROANIEL	No
5	WLSADOOTEEECEAEECRFINVEDR	No

Table 5: Test of different rail counts on the cipher text

3.3 (b) Critical Evaluation – Why the Method Is Vulnerable

#	Weakness	Why it matters today
1	Tiny key space	The only secret is the number of rails (2 to n−1). For this 25-letter message there are just 23 possible keys; all of them were tried above in under a millisecond.
2	Letter frequencies are unchanged	Transposition only moves letters. The cipher text still has English statistics (E appears 7 of 25 times ≈ 28 %), so an analyst instantly knows it is a transposition cipher, not a substitution cipher.
3	No confusion / diffusion	Shannon's principles are not met: each plaintext letter maps to a fixed, predictable position, and changing one letter changes only one cipher letter.
4	Visible structure & known-plaintext attack	Common words (THE, ATTACK, FLEE) or one known sentence let an attacker confirm the rail count at once; the zig-zag grid exposes the pattern.
5	Security by obscurity	The method itself is the secret. Modern security follows Kerckhoffs's principle: only the key may be secret, and the algorithm must stay safe even when public.
6	No integrity or authentication	Anyone can alter or replay the cipher text. Modern systems need tamper detection (MACs / AEAD).
7	Message length leaks	Cipher text length equals plaintext length, and no randomness is used, so identical messages always give identical cipher text.

Table 6: Weaknesses of the rail-fence (transposition) method

Overall, a modern computer needs well under a millisecond to break this system, and the attacker does not even need the key. The method gives obscurity, not confidentiality, and it is unsuitable for any modern communication channel.

3.4 (c) Proposed Enhancement – Replace the Legacy Layer with AES-256-GCM

The most practical improvement is to protect each message with AES-256 in Galois/Counter Mode (GCM), a standard authenticated-encryption (AEAD) algorithm. It gives confusion and diffusion, a huge key space, and an integrity tag in a single step, and it is available in every programming language and in TLS.

Step	Action
1	Generate a random 256-bit key and share it securely with each member (RSA / ECDH key exchange or an organisation key vault).
2	For every message, create a fresh 96-bit random nonce.
3	Encrypt with AES-256-GCM: cipher text + authentication tag are produced together.
4	Send nonce + cipher text + tag. The receiver decrypts; if the tag does not verify, the message is rejected.
5	(Optional, for legacy compatibility) keep the rail-fence stage inside the AES layer so old systems keep working while the real security comes from AES.

Table 7: Deployment steps for the enhancement

Demonstration (Python, cryptography library) – the same message encrypted and decrypted successfully:

```
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import os
key   = AESGCM.generate_key(bit_length=256)   # shared secret key
nonce = os.urandom(12)                        # new for every message
```

```
ct    = AESGCM(key).encrypt(nonce, b"WE ARE DISCOVERED FLEE AT ONCE", None)
print(AESGCM(key).decrypt(nonce, ct, None))    # b'WE ARE DISCOVERED FLEE AT ONCE'
```

Property	**Legacy Rail Fence (3 rails)**	**Proposed AES-256-GCM**
Key space	≈ 23 keys (for 25 letters)	2^256 keys
Letter statistics	Preserved (E = 28 %)	Cipher text looks random
Integrity check	None	128-bit authentication tag
Same message twice	Same cipher text	Different (random nonce)
Time to brute-force	< 1 millisecond	Computationally infeasible

Table 8: Legacy rail fence versus the proposed AES-256-GCM

4. Summary of Results

	Question 1 – Event Communication	**Question 5 – Legacy Encryption**
Cipher type	Caesar (mono-alphabetic substitution)	Rail Fence (transposition)
Key	Shift = 4	3 rails
Input	MEET AT NOON	WECRLTEERDSOEEFEAOCAIVDEN
Output	**QIIX EX RSSR**	**WE ARE DISCOVERED FLEE AT ONCE**
Verified by	Decrypt → original text	Re-encrypt → identical cipher text
Main weakness	25 keys; word lengths visible	23 keys; frequencies preserved

Table 9: Final results for both questions

5. Conclusion

Question 1 showed how a shift of 4 turns MEET AT NOON into QIIX EX RSSR and how the key holder reverses it with modular subtraction. Question 5 showed how a 3-rail transposition is reversed to reveal “WE ARE DISCOVERED. FLEE AT ONCE.”. Both classical ciphers have very small key spaces and leak statistical information, so they are easily broken. They remain valuable for understanding substitution and transposition, the two building blocks that modern ciphers such as AES combine on a much larger scale. For real confidentiality the legacy method should be replaced with an authenticated cipher such as AES-256-GCM.

Mapping to the Assessment Rubric

Rubric criterion (weight)	**Where it is addressed in this report**
Understanding of Concepts (25)	Concept sections for each question: substitution vs transposition, modular arithmetic, zig-zag arrangement, Shannon's principles, Kerckhoffs's principle, AEAD.
Experimental Design (30)	Rule-based methods, step-by-step tables, brute-force testing of all keys, cross-verification by reverse operation.
Use of Tools (20)	Python 3 implementation (Appendix A), systematic test runs, AES-256-GCM using the Python cryptography library.
Analysis of Results (15)	Results & Analysis sections, vulnerability table, comparison table, summary of results.
Documentation (10)	Structured headings, alphabet / rail / step tables, labelled figures, complete source code and output.

Appendix A – Python Source Code (ciphers.py)

```
import string
A = string.ascii_uppercase


# ---------- Q1 : Caesar (shift) cipher ----------
def caesar(text, k):
    """Shift every letter by k (negative k decrypts). Non-letters are kept."""
    return "".join(A[(A.index(c) + k) % 26] if c in A else c for c in text.upper())


# ---------- Q5 : Rail Fence (3-level zig-zag transposition) ----------
def rail_pattern(n, rails):
    row, step, pat = 0, 1, []
    for _ in range(n):
        pat.append(row)
        if row == 0: step = 1
        elif row == rails - 1: step = -1
        row += step
    return pat


def rail_encrypt(pt, rails):
    pat = rail_pattern(len(pt), rails)
    return "".join(pt[i] for r in range(rails) for i in range(len(pt)) if pat[i] == r)


def rail_decrypt(ct, rails):
    pat = rail_pattern(len(ct), rails)
    order = [i for r in range(rails) for i in range(len(ct)) if pat[i] == r]
    pt = [""] * len(ct)
    for ch, pos in zip(ct, order):
        pt[pos] = ch
    return "".join(pt)


if __name__ == "__main__":
    # Q1
    msg = "MEET AT NOON"
    enc = caesar(msg, 4)
    print("Q1 encrypt :", enc)
    print("Q1 decrypt :", caesar(enc, -4))
    # Q5
    ct = "WECRLTEERDSOEEFEAOCAIVDEN"
    for rails in range(2, 6):
        print(f"Q5 rails={rails}:", rail_decrypt(ct, rails))
    plain = rail_decrypt(ct, 3)
    print("Q5 plaintext :", plain)
    print("Q5 re-encrypt matches:", rail_encrypt(plain, 3) == ct)
```

Appendix B – Program Output

```
Q1 encrypt : QIIX EX RSSR
Q1 decrypt : MEET AT NOON
Q5 rails=2: WEEFCERALOTCEAEIRVDDSEONE
Q5 rails=3: WEAREDISCOVEREDFLEEATONCE
Q5 rails=4: WTEVFEEEEDARCDOECSROANIEL
Q5 rails=5: WLSADOOTEEECEAEECRFINVEDR
Q5 plaintext : WEAREDISCOVEREDFLEEATONCE
Q5 re-encrypt matches: True
```